

Laporan Praktikum Kontrol Cerdas

Week 6

Nama : Muhammad Maha Dellaroca

NIM 224308015

Kelas : TKA-6A

Akun Github (Tautan) : <https://github.com/Dellaroca>

Student Lab Assistant : Muhammad Fatih Anfa Ahima (214308092)

1. Judul Percobaan

Canny Edge Detection & Lane Detection with Instance Segmentation

2. Tujuan Percobaan

Tujuan dari percobaan praktikum Canny Edge Detection & Lane Detection with Instance Segmentation yaitu:

1. Memahami konsep Canny Edge Detection sebagai metode dasar deteksi
2. Menggabungkan metode Canny Edge Detection dengan Instance Segmentation untuk meningkatkan deteksi jalur.
3. Menggunakan Instance Segmentation untuk deteksi jalur rel kereta (Lane Detection).

3. Landasan Teori

Canny Edge Detection atau deteksi tepi adalah langkah penting dalam proses pengolahan citra yang bertujuan untuk mengidentifikasi batas-batas atau kontur objek dalam gambar, seperti Sobel, Prewitt, dan Roberts telah lama digunakan dalam dunia pengolahan citra. Namun, metode Canny Edge Detection dianggap lebih unggul karena mampu mendeteksi tepi dengan akurasi yang lebih tinggi dan menghasilkan gambar dengan tingkat kebisingan yang rendah (Luh Putu Risma Noviana dkk., 2024). Keunggulan Canny Edge Detection dibandingkan dengan deteksi tepi lainnya yaitu good detection, memaksimalkan Signal to Noise Ration (SNR) agar semua tepi dapat terdeteksi dengan baik, kemudian good location, untuk meminimalkan jarak deteksi tepi yang sebenarnya dengan tepi yang dihasilkan melalui pemrosesan, sehingga lokasi tepi terdeteksi menyerupai tepi secara nyata (semakin besar nilai Loc, maka semakin besar kualitas deteksi yang dimiliki), selanjutnya ada one respon to single edge, untuk menghasilkan tepi tunggal /tidak memberikan tepi yang bukan tepi sebenarnya (Perangin-angin dkk., 2019). Pada praktikum ini juga menggunakan proses Instance Segmentation untuk deteksi jalur rel kereta (Lane Detection). Instance Segmentation atau proses anotasi merupakan proses pemberian label pada suatu gambar, biasanya menggunakan tools Roboflow (Wulanningrum dkk., 2024). Pelatihan datanya menggunakan kurang lebih 2000 foto rel kereta api dengan berbagai macam track, ada yang lurus, berbelok, single track, double track, dan lain-lain. Setiap epoch melibatkan pemasukan dataset ke dalam model, penyesuaian parameter model, dan pembaruan bobot untuk mengoptimalkan kinerja model dalam mendeteksi dan segmentasi rel kereta api dengan akurat. Proses iteratif ini bertujuan untuk meminimalkan perbedaan antara rel kereta api yang diprediksi dengan objek lain dan anotasi kebenaran dasar dalam dataset pelatihan. Teknik augmentasi data diterapkan selama pelatihan untuk meningkatkan kemampuan generalisasi model (Husain dkk., 2024). Algoritma ini tahan terhadap oklusi yang kompleks, sehingga sangat berguna untuk adaptasi terhadap lingkungan AV. Algoritma ini telah digabungkan oleh arsitektur jaringan klasifikasi seperti ENet, Segnet, dan Resnet38 yang dibuat khusus untuk identifikasi gambar. Hasilnya adalah arsitektur Resnet38 memiliki akurasi rata-rata yang paling tinggi, namun membutuhkan memori yang lebih besar (Darwin & Saputro, 2021).

4. Analisis dan Diskusi

Analisis

A. Perbandingan Canny Edge Detection dan Instance Segmentation dalam Deteksi Jalur

1. Canny Edge Detection

- Sensitif terhadap noise, sehingga bisa menghasilkan banyak tepi yang tidak relevan.
- Tidak memahami konteks, hanya mendeteksi perbedaan warna atau cahaya.
- Kurang efektif dalam kondisi jalan yang kompleks, seperti persimpangan atau cahaya yang berubah-ubah.

2. Instance Segmentation

- Memahami konteks gambar sehingga bisa membedakan jalur dengan lebih baik.
- Lebih tahan terhadap noise dan perubahan pencahayaan.
- Bisa mengenali lebih dari satu objek sekaligus, misalnya beberapa jalur dalam satu gambar.
- Lebih baik dalam menangani kondisi jalan yang rumit.

B. Kombinasi Canny Edge Detection dan Instance Segmentation

Menggabungkan kedua metode ini bisa meningkatkan akurasi deteksi jalur.

- Canny Edge Detection berguna untuk mendeteksi tepi dengan cepat, sementara Instance Segmentation memberikan pemahaman yang lebih baik terhadap jalur.
- Dalam skenario yang sangat kompleks (seperti persimpangan besar), Instance Segmentation tetap lebih dominan, sedangkan Canny bisa membantu dalam mendeteksi detail kecil.
- Supaya kombinasi ini bekerja optimal, perlu dilakukan eksperimen dan penyetelan parameter agar bisa beradaptasi dengan berbagai kondisi jalan.

C. Pengaruh Parameter pada Canny Edge Detection

Canny Edge Detection bergantung pada dua parameter utama:

1. **Lower Threshold (T1):** Jika terlalu rendah, banyak noise akan ikut terdeteksi. Jika terlalu tinggi, beberapa jalur bisa hilang.
2. **Upper Threshold (T2):** Jika terlalu tinggi, hanya jalur yang sangat jelas yang terdeteksi. Jika terlalu rendah, hasilnya bisa terlalu banyak garis yang tidak relevan.
3. Piksel yang ada di antara kedua threshold akan dipertahankan hanya jika terhubung dengan tepi yang kuat (hysteresis thresholding).

Diskusi

A. Kapan Canny Edge Detection Lebih Baik daripada Instance Segmentation?

Canny Edge Detection masih berguna dalam beberapa situasi:

- Jika keterbatasan perangkat keras tidak memungkinkan penggunaan model yang lebih kompleks.
- Jika lingkungan cukup sederhana tanpa banyak gangguan.
- Jika dibutuhkan deteksi yang sangat cepat dengan sumber daya terbatas. Namun, metode ini tidak efektif dalam kondisi jalan yang kompleks, banyak gangguan, atau jika diperlukan deteksi multi-objek.

B. Cara Meningkatkan Deteksi Jalur dengan YOLOv8-seg

Agar deteksi jalur dengan YOLOv8-seg lebih akurat, beberapa strategi dapat digunakan:

- **Optimalkan learning rate** agar model tidak overfitting atau underfitting, misalnya dengan **Cosine Annealing Scheduler**.
- **Sesuaikan batch size** dengan kapasitas GPU; gunakan batch besar untuk stabilitas atau **gradient accumulation** jika memori terbatas.
- **Atur anchor box** dengan algoritme **k-means clustering** agar lebih sesuai dengan ukuran jalur.
- **Gunakan resolusi gambar lebih tinggi** jika jalur terlalu tipis untuk dideteksi dengan jelas.
- **Terapkan data augmentation** seperti flipping, penyesuaian kecerahan, dan transformasi perspektif agar model lebih fleksibel dalam berbagai kondisi.
- **Optimalkan loss function** agar model lebih fokus pada jalur utama dan mengabaikan gangguan.
- **Gunakan transfer learning** dari model yang sudah dilatih sebelumnya untuk mempercepat proses pelatihan dan meningkatkan akurasi.

C. Manfaat YOLOv8-seg untuk Navigasi Kereta Otomatis

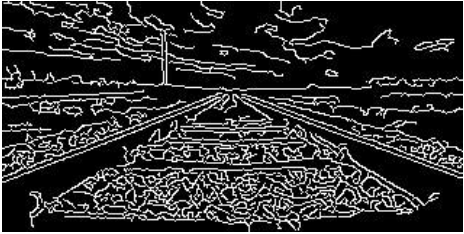
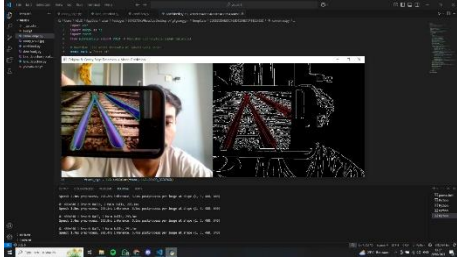
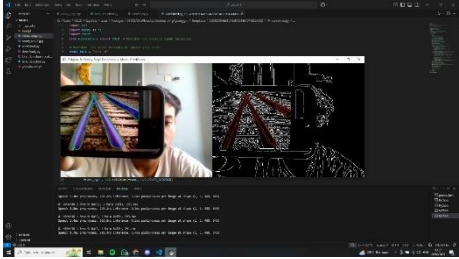
Penerapan YOLOv8-seg dalam navigasi kereta otomatis memiliki banyak manfaat:

- **Deteksi jalur rel yang akurat dan real-time**, memastikan kereta tetap berada di lintasan yang benar.
- **Mampu mengenali percabangan rel dan rintangan** seperti benda jatuh atau pejalan kaki.
- **Menyesuaikan kecepatan kereta secara otomatis** demi keselamatan.
- **Dapat beradaptasi dengan kondisi lingkungan yang berbeda**, misalnya perubahan pencahayaan dan cuaca ekstrem.
- **Hasil deteksi diperbaiki dengan post-processing**, seperti **Non-Maximum Suppression (NMS)** untuk menghindari deteksi ganda dan **morphological operations** agar segmentasi jalur lebih halus.
- **Perlu dilakukan uji lapangan** untuk memastikan model bekerja dengan baik di dunia nyata, dengan fokus pada akurasi, recall, precision, dan latency.

5. Assignment

Berdasarkan praktikum keenam yang telah dilakukan dari kode program ini adalah membuat sebuah sistem yang dapat melakukan deteksi tepi menggunakan metode Canny Edge Detection dan mengombinasikannya dengan deteksi objek atau segmentasi menggunakan model YOLO dari Ultralytics untuk menyoroti objek yang terdeteksi dalam sebuah video yang diambil secara real-time dari kamera. Program dimulai dengan memuat model YOLO dari path yang telah ditentukan menggunakan YOLO(MODEL_PATH), memastikan bahwa model tersedia sebelum melanjutkan ke pemrosesan video menggunakan OpenCV. Setelah kamera diakses melalui cv2.VideoCapture(0), setiap frame yang ditangkap dikonversi ke skala abu-abu dengan cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) agar dapat dilakukan deteksi tepi menggunakan metode Canny Edge Detection, yang menghasilkan gambar biner dengan tepi putih pada latar belakang hitam melalui cv2.Canny(gray, low_threshold, high_threshold). Secara paralel, frame asli dikonversi ke format RGB dengan cv2.cvtColor(frame, cv2.COLOR_BGR2RGB), kemudian diproses oleh model YOLO untuk melakukan prediksi objek atau segmentasi, di mana hasil prediksi pertama disimpan dalam predictions = results[0]. Jika model menggunakan segmentasi, maka program akan menggambar area objek menggunakan poligon biru transparan dengan cv2.fillPoly(overlay, [mask], (255, 0, 0)), sedangkan jika model hanya mendeteksi objek dalam bentuk bounding box, maka objek akan dilingkari dengan kotak hijau menggunakan cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2), serta label objek akan ditampilkan jika tersedia menggunakan cv2.putText(). Selanjutnya, masking digunakan untuk membedakan tepi objek yang terdeteksi oleh YOLO dengan tepi yang dideteksi oleh metode Canny, di mana tepi dalam area objek yang dikenali YOLO akan diwarnai merah, sementara tepi lainnya tetap berwarna putih dengan edges_bgr[np.where((mask_model == 255) & (edges == 255))] = [0, 0, 255]. Kedua hasil, yaitu frame dengan anotasi YOLO dan hasil deteksi tepi, ditampilkan secara berdampingan menggunakan np.hstack() untuk memberikan tampilan visual yang lebih jelas mengenai perbedaan antara tepi umum dan tepi dari objek yang dikenali. Program berjalan dalam loop hingga pengguna menekan tombol 'q', yang akan memicu cv2.waitKey(1) & 0xFF == ord('q') untuk keluar, setelah itu kamera akan dilepaskan dengan cap.release() dan semua jendela yang terbuka akan ditutup dengan cv2.destroyAllWindows() untuk mencegah kebocoran memori. Dengan kombinasi dari metode deteksi tepi tradisional dan teknologi deep learning untuk segmentasi atau deteksi objek, program ini dapat digunakan untuk berbagai aplikasi seperti deteksi rel, pengawasan lalu lintas, dan lainnya.

6. Data dan Output Hasil Pengamatan

No	Variabel	Hasil Pengamatan	
1	Canny Edge Detection		
2	Lane Detection		
3	Combined		

7. Kesimpulan

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat diambil kesimpulan yaitu:

1. Teknik masking digunakan untuk membedakan tepi objek yang terdeteksi YOLO dengan tepi umum dari hasil Canny Edge Detection, memungkinkan visualisasi yang lebih jelas.
2. Program berhasil mengombinasikan metode Canny Edge Detection dengan model YOLO untuk mendeteksi tepi sekaligus mengenali dan menyoroti objek dalam video real-time.
3. Program dapat berjalan dalam loop hingga pengguna menekan tombol keluar, dengan pengelolaan memori yang baik untuk menghindari kebocoran sumber daya.

8. Saran

Untuk meningkatkan performa dan akurasi, program dapat dioptimalkan dengan penggunaan GPU agar prediksi YOLO berjalan lebih cepat dan responsif. Selain itu, preprocessing tambahan seperti Gaussian Blur sebelum Canny Edge Detection dapat membantu mengurangi noise yang tidak diperlukan. Untuk meningkatkan keterbacaan visual, transparansi overlay warna dapat disesuaikan agar objek yang terdeteksi lebih jelas tanpa menutupi detail penting dalam gambar. Menambahkan fitur penyimpanan hasil deteksi dalam bentuk gambar atau video bisa menjadi nilai tambah untuk analisis lebih lanjut di berbagai aplikasi dunia nyata.

9. Daftar Pustaka

Husain, B. H., Osawa, T., Astiti, S. P. C., Frianto, D., Nandika, M. R., & Agustine, D. (2024). Deteksi Lubang Jalan Secara Otomatis dari Rekaman Drone Menggunakan Model Machine Learning Berbasis YOLOv5 Instance Segmentation di Kota Pekanbaru, Provinsi Riau, Indonesia. *Jurnal Zona*, 8(2), 137–146.

Luh Putu Risma Noviana, I Putu Eka Indrawan, & Gde Iwan Setiawan. (2024). ANALYSIS OF CANNY EDGE DETECTION METHOD FOR FACIAL RECOGNITION IN DIGITAL IMAGE PROCESSING. *Jurnal Manajemen dan Teknologi Informasi*, 15(2), 29–34.
